

SIMATS Engineering College — Department of Information Technology

Course: Object Oriented Analysis and Design | Code: CSA1142 | CO4 | Marks: 40

Assessment Tool 2 — Industry Problem Solving Task | Q.No.: Q4

Question: Analyze how object-oriented design principles and patterns are applied in this system, and explain how the layered architecture improves scalability and integration with the database.

ANSWER

1. Introduction

The Smart Inventory Management System (SIMS) for a retail chain is built on solid Object-Oriented (OO) design principles and well-known design patterns. It employs a three-layered architecture — View Layer, Business Layer, and Access Layer — to ensure separation of concerns, scalability, and seamless integration with an Object-Oriented Database Management System (OODBMS).

2. Object-Oriented Design Principles Applied

Encapsulation: Each layer encapsulates its own logic and data. The Business Layer hides inventory processing rules from the View Layer, exposing only well-defined interfaces. Store objects encapsulate stock levels and alert thresholds internally.

Abstraction: Abstract interfaces are defined for inventory policies (Strategy pattern) and database access, allowing the rest of the system to work with abstractions rather than concrete implementations. This makes the system easier to extend without modifying existing code.

Inheritance: Different inventory policy classes (e.g., FIFO, LIFO, JIT) inherit from a common abstract InventoryPolicy base class, enabling polymorphic behavior when the Strategy pattern switches between policies at runtime.

Polymorphism: The Strategy pattern leverages polymorphism: the Business Layer calls a common applyPolicy() method, and the correct subclass implementation executes at runtime depending on the store configuration.

Single Responsibility Principle (SRP): Each class has a single responsibility — the View Layer handles display, the Business Layer processes logic, and the Access Layer manages database communication. This minimises the impact of changes.

Open/Closed Principle (OCP): New inventory strategies can be added as new classes without modifying existing business logic, keeping the system open for extension but closed for modification.

3. Design Patterns Applied

3.1 Strategy Pattern

The Strategy pattern is used to manage different inventory policies (FIFO, LIFO, Just-In-Time, etc.) dynamically. A context class (e.g., InventoryManager) holds a reference to an InventoryPolicy interface. At runtime, the appropriate concrete strategy is injected based on the store's requirements. This allows the retail chain to apply different restocking policies per store without changing the core business logic, greatly improving flexibility and maintainability.

Component	Role
InventoryPolicy (Interface)	Defines applyPolicy() contract
FIFOPolicy	Implements first-in-first-out restocking
LIFOPolicy	Implements last-in-first-out restocking

JITPolicy	Implements just-in-time restocking
InventoryManager (Context)	Delegates to selected policy at runtime

3.2 Singleton Pattern

The Singleton pattern is used for centralized configuration management. A single instance of ConfigurationManager is shared across the entire application, ensuring consistent configuration values (e.g., alert thresholds, database connection settings) without redundant object creation. This prevents conflicting configurations, reduces memory overhead, and provides a single point of control for system-wide settings.

Benefit	Explanation
Single Instance	Only one ConfigurationManager exists throughout the app lifecycle
Global Access	Any layer can access configuration via getInstance()
Thread Safety	Controlled instantiation prevents race conditions in multi-store updates
Resource Efficiency	Avoids redundant DB connection objects and configuration reloads

4. Layered Architecture and Its Benefits

4.1 View Layer (Presentation Layer)

The View Layer is responsible only for displaying dashboards showing stock levels, alerts, and reports across stores. It communicates exclusively with the Business Layer through defined interfaces, never directly accessing the database. This separation means UI changes do not affect business logic or data access, improving maintainability.

4.2 Business Layer (Logic Layer)

The Business Layer processes inventory updates, applies the Strategy pattern to enforce inventory policies, and generates alerts when stock levels breach thresholds. It acts as the mediator between the View and Access layers. By centralizing business rules here, any change in policy (e.g., switching to JIT) only affects this layer, without touching the UI or database code — a key scalability advantage.

4.3 Access Layer (Data Access Layer)

The Access Layer is the only component that communicates with the OODBMS. It uses object-oriented query optimizations to retrieve and persist inventory objects efficiently. Abstracting database interactions here means the system can switch from one OODBMS to another (or to a relational DB) by modifying only this layer, without impacting business or UI logic.

5. How Layered Architecture Improves Scalability

- **Independent Scaling:** Each layer can be scaled independently. If inventory processing becomes a bottleneck, more Business Layer instances can be deployed without changing the View or Access layers.
- **Loose Coupling:** Layers communicate through interfaces, reducing dependencies. Adding a new store or inventory category requires changes in only the relevant layer.
- **Parallel Development:** Different teams can develop the View, Business, and Access layers simultaneously, accelerating delivery for future expansion.
- **Reusability:** The Business Layer logic can be reused by different front-end clients (web, mobile, kiosk) without duplication.
- **Load Distribution:** Database load is managed centrally through the Access Layer, enabling connection pooling and query optimization strategies.

6. Integration with OODBMS

The use of an OODBMS complements the object-oriented design by storing inventory objects (e.g., StockItem, Store, Alert) directly as objects rather than mapping them to relational tables. This eliminates the impedance mismatch seen in relational databases and allows complex object graphs (e.g., a Store containing multiple StockItems with nested Supplier references) to be retrieved efficiently in a single query. The Access Layer leverages OODBMS features like object identity, inheritance-based queries, and persistent object references to achieve high-performance data retrieval, directly supporting the system's performance goals.

7. Conclusion

The Smart Inventory Management System effectively applies OO design principles (encapsulation, abstraction, inheritance, polymorphism, SRP, OCP) and key design patterns (Strategy for flexible policy management, Singleton for centralized configuration). The three-layered architecture ensures clean separation of concerns, makes the system scalable for future multi-store expansion, and facilitates seamless, high-performance integration with the OODBMS. Together, these design choices produce a robust, maintainable, and extensible inventory management solution.